

Max Tokens

Sets a hard limit on the total length of the AI's response, measured in tokens. This parameter is crucial for API cost control and preventing runaway generation.

Example: `max_tokens = 100` → model stops after ~100 tokens even if the response is incomplete.

Use higher values for detailed answers and lower values for concise responses or tight budgets.

<> OPENAI & GEMINI EXAMPLES

OPENAI

```
response = client.chat.completions.create(
    model="gpt-4o",
    max_tokens=150
)
print(response.choices[0].message.content)
```

GOOGLE GEMINI

```
response = client.models.generate_content(
    model="gemini-2.0-flash",
    config={"max_output_tokens": 150}
)
print(response.text)
```

Controls randomness of AI responses. Lower values = more focused, higher values = more creative.

Low temperature (0.0 – 0.3): great for fact-based tasks, code generation, and data extraction.

High temperature (0.8 – 1.5): better for brainstorming, storytelling, and creative writing.

<> OPENAI & GEMINI EXAMPLES

OPENAI

```
response = client.chat.completions.create(  
    model="gpt-4o",  
    temperature=0.7  
)  
print(response.choices[0].message.content)
```

GOOGLE GEMINI

```
response = client.models.generate_content(  
    model="gemini-2.0-flash",  
    config={"temperature": 0.7}  
)  
print(response.text)
```

Top-P (Nucleus Sampling)

Filters the model's vocabulary based on cumulative probability (nucleus sampling). The model selects tokens from the smallest possible set whose combined probability equals P.

`top_p = 0.9` → only the most likely words adding up to 90% probability are considered.

Helps avoid bizarre, low-probability tokens while keeping outputs coherent and flexible.

<> OPENAI & GEMINI EXAMPLES

OPENAI

```
response = client.chat.completions.create(  
    model="gpt-4o",  
    top_p=0.9  
)  
print(response.choices[0].message.content)
```

GOOGLE GEMINI

```
response = client.models.generate_content(  
    model="gemini-2.0-flash",  
    config={"top_p": 0.9}  
)  
print(response.text)
```

Limits the model to consider only the top K most likely tokens at each step. Unlike top-p which uses probability mass, top-k uses a fixed number.

Lower K values produce more focused, deterministic outputs; higher values allow more diversity.

Often combined with top-p for fine-grained control over randomness.

<> OPENAI & GEMINI EXAMPLES

OPENAI

```
response = client.chat.completions.create(  
    model="gpt-4o",  
    top_k=50  
)  
print(response.choices[0].message.content)
```

GOOGLE GEMINI

```
response = client.models.generate_content(  
    model="gemini-2.0-flash",  
    config={"top_k": 50}  
)  
print(response.text)
```


Penalizes tokens based on how often they appear in the generated text. Higher values reduce repetition of words that have already been used.

Range: -2.0 to 2.0. Positive values reduce repetition; negative values encourage it.

Useful to prevent looping and make long responses feel more natural.

<> OPENAI & GEMINI EXAMPLES

OPENAI

```
response = client.chat.completions.create(  
    model="gpt-4o",  
    frequency_penalty=1.0  
)  
print(response.choices[0].message.content)
```

GOOGLE GEMINI

```
response = client.models.generate_content(  
    model="gemini-2.0-flash",  
    config={"frequency_penalty": 1.0}  
)  
print(response.text)
```

Penalizes tokens if they appear at all in the generated text, regardless of frequency.

Encourages the model to introduce new topics and vocabulary instead of staying stuck on the same idea.

Works together with `frequency_penalty` for comprehensive repetition and topic control.

<> OPENAI & GEMINI EXAMPLES

OPENAI

```
response = client.chat.completions.create(
    model="gpt-4o",
    presence_penalty=1.0
)
print(response.choices[0].message.content)
```

GOOGLE GEMINI

```
response = client.models.generate_content(
    model="gemini-2.0-flash",
    config={"presence_penalty": 1.0}
)
print(response.text)
```

Strings that act as termination signals. When the model encounters any stop sequence, it immediately stops generating—even if `max_tokens` has not been reached.

Critical for structured outputs, API responses, or chatbots where you must not let the model "keep talking" forever.

Examples: stop at `"\n"` for single-line output, or stop at `"User:" / "###"` to prevent roleplay loops.

<> OPENAI & GEMINI EXAMPLES

OPENAI

```
response = client.chat.completions.create(
    model="gpt-4o",
    stop=["###", "User:"]
)
print(response.choices[0].message.content)
```

GOOGLE GEMINI

```
response = client.models.generate_content(
    model="gemini-2.0-flash",
    config={"stop_sequences": ["###"]}
)
print(response.text)
```

The number of tokens in your input prompt. Tokens are chunks of text (not always full words).

Token count impacts cost, latency, and whether you fit inside the model's context window.

Rule of thumb: 1 token $\approx$ 4 characters of English text.

<> OPENAI & GEMINI EXAMPLES

OPENAI

```
import tiktoken

# Count tokens in a message
enc = tiktoken.encoding_for_model("gpt-4o")
text = "Your input text here"
tokens = enc.encode(text)
token_count = len(tokens)

print(f"Number of tokens: {token_count}")
```


The maximum number of tokens (input + output) that a model can process in a single request.

If you exceed the limit, the API will error or silently drop parts of your prompt.

Always leave headroom for the model's reply: context window = input tokens + output tokens.

<> OPENAI & GEMINI EXAMPLES

OPENAI

```
# GPT-4o: 128k context window
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "user", "content": "Your prompt here"}
    ],
    max_tokens=4000 # Reserve tokens for output
)

print(f"Total tokens: {response.usage.total_tokens}")
```

Master these 9 parameters and you'll control cost, creativity, determinism, and reliability across OpenAI and Gemini. Save this for your next production build.
